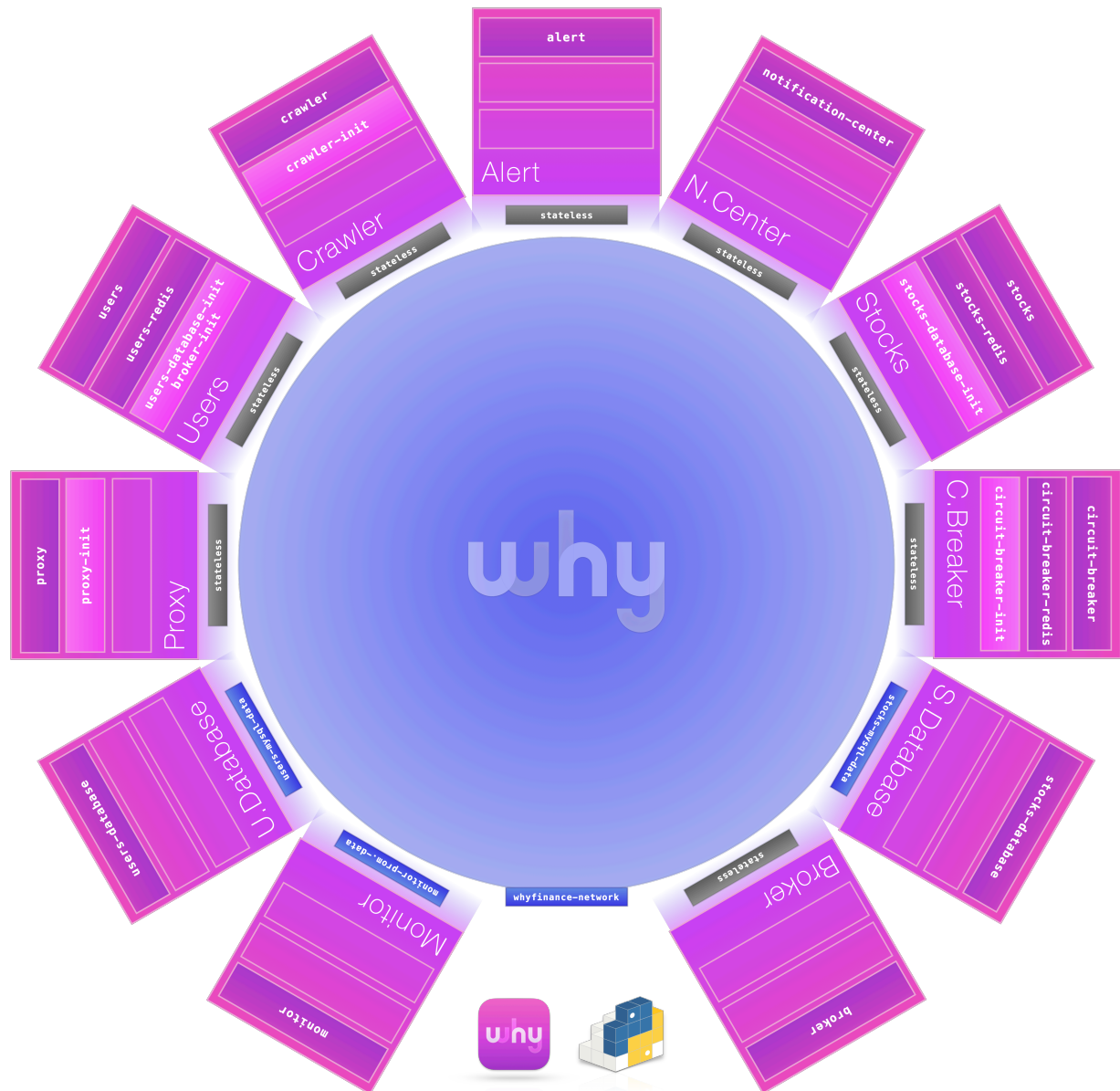


Why Finance

Homework 3



Le novità



Abbiamo portato la nostra soluzione su Kubernetes e abbiamo introdotto il monitoraggio dei servizi Proxy e Crawler basandoci sulla piattaforma Prometheus

Con la consegna di Homework 3 abbiamo ulteriormente esteso l'architettura di Why Finance portando la nostra soluzione su Kubernetes e abbiamo introdotto il monitoraggio dei servizi Proxy e Crawler basandoci sulla piattaforma Prometheus. Per Kubernetes abbiamo scelto di utilizzare l'implementazione Kind (Kubernetes in Docker), in modo da poter agevolmente creare e gestire un cluster locale limitando il consumo di risorse di soluzioni basate su VM, o la complessità di una soluzione cloud.

Il cluster Kind

Ogni micro-servizio, in precedenza definito singolarmente nel `docker-compose.yml`, è stato tradotto in un apposito file di manifest Kubernetes (in formato YAML): in particolare quasi ogni manifest include: un Deployment che definisce le modalità di avvio e gestione del container e un Service che ne configura l'esposizione della rete e la comunicazione con il micro-servizio; grazie a quest'ultimo, ciascun container conserva il medesimo hostname e le proprie dipendenze, ma è orchestrato da Kubernetes che ne gestisce il ciclo di vita, la scalabilità e i riavvii in caso di errori.

Per la creazione e la configurazione del cluster Kind locale, abbiamo predisposto un file chiamato `kind-config.yaml`, definito come segue:

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: why-finance
nodes:
- role: control-plane
  extraMounts:
  - hostPath: "${WHYFINANCE_PATH}"
    containerPath: /why-finance
- role: worker
  labels:
    custom-label: "worker"
  extraMounts:
  - hostPath: "${WHYFINANCE_PATH}"
    containerPath: /why-finance
  extraPortMappings:
  - containerPort: 30080 # Proxy (Flask)
    hostPort: ${PROXY_PORT} # Defaults to 80
  - containerPort: 30090 # Monitor (Prometheus)
    hostPort: 9090
```

Uno degli ostacoli principali riscontrati riguardava l'impossibilità da parte dei nodi del cluster di accedere all'interezza dei file sorgente del progetto `why-finance`, come `prometheus.yml` (che viene inserito nel rispettivo container per *bind-mount* del singolo file), presenti sul sistema host (analogamente al fatto che le immagini Docker vanno caricate manualmente su tutti i nodi Kind e non sono immediatamente accessibili).

Questo limite è stato superato utilizzando la direttiva `extraMounts`, che ha permesso di montare l'intera directory del progetto locale nel percorso `/why-finance` all'interno dei nodi del cluster, che per semplicità abbiamo limitato a un nodo `control-plane` e un nodo `worker`.

Infine notiamo la direttiva `extraPortMappings`, che consente di accedere dall'esterno del sistema al Proxy per comunicare con esso, e al Monitor Prometheus per osservarne le metriche, a loro volta configurati con `NodePort` invece di `ClusterIP` nel Deployment dei loro rispettivi manifest.

Monitoraggio con Prometheus

Come richiesto nella consegna di Homework 3, abbiamo inserito il monitoraggio mediante la piattaforma Prometheus dei micro-servizi, in particolare di Crawler (in quanto operazione di elaborazione batch continua) e di Proxy (da cui partono le richieste a tutti i componenti del sistema e quindi permette di saggiare indirettamente il funzionamento e le prestazioni di Database, Redis e servizi User e Stocks).

In accordo al refactor eseguito in Homework 2, anche il monitoraggio è supportato da una libreria personalizzata `utils.monitor` che automatizza l'assegnazione delle etichette richieste del nome del servizio e del nodo Kubernetes su cui sono in esecuzione (rispettivamente fornite tramite variabili d'ambiente (`$MONITOR_SERVICE_NAME` e `$MONITOR_SERVICE_NODE`, quest'ultima sostituita da una stringa statica quando si esegue in Docker Compose).

In entrambi i servizi si monitora una metrica Gauge e una Counter, che per Crawler sono durata di un ciclo di monitoraggio e numero di cicli completati, mentre per Proxy sono tempo per soddisfare una richiesta e numero di richieste servite: in particolare per Proxy, etichette aggiuntive sono applicate per il metodo HTTP, il percorso e il codice HTTP di ritorno, consentendo ad esempio di filtrare le metriche relative ad uno specifico servizio (`/users` oppure `/stocks`).

Quest'ultima operazione è possibile attraverso il container apposito di Prometheus, che prende il nome di `Monitor`, disponibile sia in configurazione su Docker Compose sia nel cluster Kubernetes su Kind alla porta 9090 per mezzo di un `Service` di tipo `NodePort`.

Lo script di installazione è stato aggiornato

L'integrazione di Kubernetes tramite Kind nello script `setup.sh` ha richiesto una nuova opzione `--kind` consente di avviare, configurare e gestire un cluster Kubernetes completamente funzionale.

Lo script automatizza l'applicazione del `kind-config.yaml` e di tutti i manifest relativi ai componenti del sistema; dopo la configurazione del cluster, lo script gestisce l'operazione di *build* delle immagini Docker dei microservizi ed il loro caricamento nei nodi del cluster di Kind.

Per garantire un uso efficiente del tempo durante i cicli di sviluppo iterativi, se le immagini vengono create utilizzando i Dockerfile solo se non sono già

presenti nel registro locale. Una volta pronte, queste immagini vengono trasferite nel cluster tramite il comando `kind load docker-image`.

Al contrario del `compose.yaml`, i manifest di Kubernetes non prevedono la sostituzione automatica delle variabili di ambiente a partire da un file `.env` apposito: per colmare questa mancanza, lo script di setup provvede, prima dell'applicazione, a processare il manifest con `envsubst`, sostituendo dinamicamente variabili come il percorso del progetto o la porta del Proxy. Questo passaggio assicura che i file siano adattati al contesto locale e pronti per essere eseguiti nel cluster, oltre a consentire di modificare porte che possono risultare fastidiose in maniera centralizzata (come la porta 80 impiegata dal proxy) sia nei confronti dei manifest Kubernetes, sia nel Docker Compose, che nei singoli sorgenti (il proxy ascolta alla porta che gli viene fornita tramite variabile d'ambiente).

Poiché il processo per mettere in funzionamento un cluster Kubernetes è molto più lungo rispetto al sistema equivalente basato su Docker Compose, lo script include una fase di verifica finale dello stato del cluster: utilizzando il comando `kubectl wait`, controlla che tutti i pod siano in stato di "Ready" prima di terminare.

Guida di configurazione

Clonare il progetto

```
git clone <https://github.com/alessiogiordano/why-finance.git>
```

Configurazione delle Variabili d'ambiente

Il file `/Containers/.env` consente di configurare in un solo posto le variabili d'ambiente come la porta cui esporre il Proxy con `PROXY_PORT` o l'intervallo fra cicli di aggiornamento dei titoli di borsa con `CRAWLER_TIME_INTERVAL`.

In Homework 3 le variabili d'ambiente sono invariate rispetto Homework 2.

```
USERS_DB_HOST=users_database
USERS_DB_PORT=3306
USERS_DB_USER=root
USERS_DB_PASSWORD=root
USERS_DB_NAME=whyfinance_users
STOCKS_DB_HOST=stocks_database
STOCKS_DB_PORT=3306
STOCKS_DB_USER=root
STOCKS_DB_PASSWORD=root
STOCKS_DB_NAME=whyfinance_stocks

PROXY_PORT=80
REDIS_PORT=6379
WATCH_SERVER_PORT=50052
USER_SERVER_PORT=50051
CIRCUIT_BREAKER_PORT=30112

CRAWLER_TIME_INTERVAL=60 # Secondi tra un ciclo e l'altro
APNS_ENDPOINT=https://api.push.apple.com:443/3/device/
KAFKA_BROKERS=broker:9092
KAFKA_TOPICS=crawler,alert,notification-center
```

Il nuovo script di installazione

Per guidarvi per mano nella configurazione del progetto abbiamo realizzato uno script di shell di installazione che in maniera automatica si prenderà cura di compilare i Protocol Buffer, posizzionarli nella cartella `.build/` per poi essere copiati a tempo di compilazione dei container dentro ciascuno di questi, nonché di copiare il certificato necessario per l'autenticazione presso APNS (che sarebbe inappropriato, per ragioni ovvie di sicurezza, lasciare all'interno del repository pubblico, ma che invece vi allegheremo insieme al messaggio di posta di consegna di Homework 2 e Homework 3), oltre a avviare i vari container e pod e a consentire varie opzioni di pulizia del contesto di lavoro.

```

why-finance % ./setup.sh -h
WhyFinance
Progetto di Distributed Systems and Big Data
Anno Accademico 2024-25
(C) 2024-2025 Luca Montera, Alessio Giordano
---
USAGE: ./setup.sh [-d | -k] [--install-protobuf-support] [--build-protos] [-h] ...
--transfer-certificate aps.pem      Copies the APNS certificate into the project
--install-protobuf-support          Installs the proper version of grpcio(-tools)
--build-protos                      Deletes ./Protos/.build and rebuilds ProtoBufs
--rebuild                          Forces rebuild of images before running
--reset                             Deletes built local images and volumes before running
--hard-reset                       Deletes all built images and volumes before running
-d --docker                        Run Docker Compose through the Docker Engine
-k --kind                          Run Kubernetes Cluster through Kind
--log-pods                         Log status of Kubernetes nodes to ./log.txt
-h, --help                         Prints this message
why-finance %

```

Poiché il sistema può essere eseguito sia come Docker Compose che come cluster Kind, l'argomento `--no-run` non è più supportato (perché il comando non esegue più il sistema a prescindere) e sono stati introdotti opportune flag `--docker` oppure `--kind` per scegliere dove eseguire il sistema.

```

why-finance % ./setup.sh --docker # Esegue tramite Docker Compose
why-finance % ./setup.sh --kind   # Crea cluster Kind (distruggendo quello preesistente)

```

Ricordiamo che è necessario trasferire il certificato per l'uso della funzionalità di invio notifiche tramite APNS, che può essere fatto con l'opportuno argomento, anche simultaneamente all'esecuzione con Docker o Kind.

```

why-finance % ./setup.sh -d --transfer-certificate ~/Desktop/aps.pem # Run as --docker
why-finance % ./setup.sh -k --transfer-certificate ~/Desktop/aps.pem # Run as --kind

```

Uso del servizio web HTTP

Poiché Homework 3 non richiedeva l'implementazione di nuove funzionalità utente, il servizio HTTP alla porta 80 è rimasto invariato:

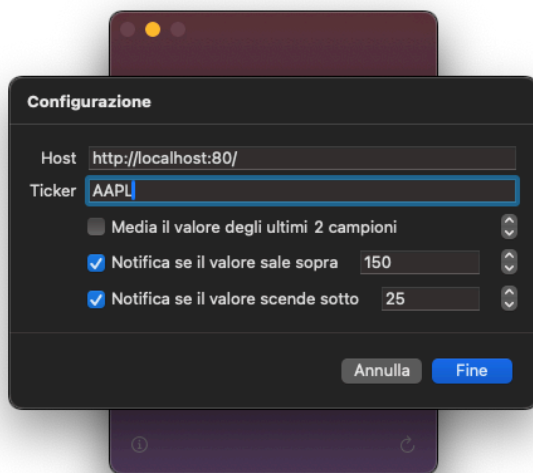
```

# Restituisce 204 No Content se il sistema è in funzione
GET /health
# Prova ad effettuare l'inserimento dell'utente e se l'operazione fallisce (l'utente esiste già) riprova aggiornando invece l'utente, restituendo 204 No Content
PUT /users/<device_id>
- BODY: <ticker> # Content-Type: text/plain
- BODY: { 'ticker':_, 'device_token':_, 'low_value':_, 'high_value':_ } # application/json
# Elimina dal database tutti i record aventi quel dato indirizzo email (per vincolo di unicità sarà al più uno solo, ma se l'indirizzo non è presente non sarà restituito alcun errore, ma sempre 204 No Content
DELETE /users/<device_id>
# Restituisce l'ultimo valore presente nella tabella 'stock_data' per quel dato ticker, o sarà lanciata una eccezione se nessun dato è presente
GET /stocks/<ticker>
# Restituisce la media degli ultimi count valori per quel dato ticker, o sarà lanciata una eccezione se nessun dato è presente
GET /stocks/<ticker>?avg=<count>

```

App WhyFinance per macOS con ricezione tramite Apple Push Notification Service

In Homework 3 l'applicazione Why Finance non ha subito modifiche, quindi per il suo utilizzo valgono le medesime considerazioni circa Homework 1 per la parte di interazione con i servizi Users e Stocks (allora quest'ultimo prendeva il nome di Watch) e Homework 2 per il meccanismo di messaggistica tramite notifiche push (ricordiamo che qui AlertSystem prende il nome semplicemente di Alert, mentre AlertNotifierSystem è NotificationCenter).



Ricordiamo in particolare è possibile trovare l'applicazione la si trova in '/Apps/Why Finance.app.zip' e che è importante usare una versione dell'app compatibile con la controparte server, quindi 1.0 per Homework 1 e 2.0 per Homework 2 e Homework 3.



È sufficiente seguire le indicazioni fornite dall'interfaccia per configurare i parametri del titolo azionario da monitorare, causando dietro le quinte l'interazione con il servizio User, che renderà disponibili i dati dei titoli da indicizzare a Crawler, che informerà Alert quando avrà terminato la sua operazione, che chiederà a NotificationCenter di rendere noto all'app, o in background se nessuna notifica esplicita è richiesta, oppure con un banner: si ricorda che è necessario abilitare

manualmente i banner o gli alert per WhyFinance dalle preferenze di sistema (in 'notifiche') su macOS se si desidera visualizzare l'arrivo delle notifiche.

